Copy Trading Platform
Product Concept & Technical Blueprint

Version 1.0 • Draft for development planning

Product scope	Universal copy trading platform connecting traders and followers across multiple exchanges
V1 exchanges	Hyperliquid, Aster, BloFin
Primary market	Perpetual futures only
Primary audience	Web3-native users seeking premium multi-exchange copy trading

This document defines the product scope, user flows, platform behavior, technical architecture, data model, billing design, copy engine logic, security requirements, and V1/V2 roadmap for a universal copy trading platform.

1. Executive Summary

The platform connects two core participants: traders and followers. Approved traders execute trades on supported exchanges. Followers subscribe to a specific trader, connect their own exchange account, and have new trader positions mirrored automatically on their selected exchange, subject to symbol availability, margin, slippage, and user-defined limits.

The core product promise is universality: the system should not be tied to a single exchange. Instead, it should normalize trade events from a trader source exchange and translate them for execution on the follower target exchange.

V1 focuses on speed, reliability, and clarity rather than maximum breadth. The first release supports perpetual futures only, supports long and short positions, and launches with Hyperliquid, Aster, and BloFin.

The product will present a premium Web3/trading-focused frontend, transparent trader statistics, in-app and Telegram notifications, wallet-based billing, and a backend architecture built for event-driven execution.

2. Product Vision

The product vision is to become a universal copy trading platform that connects traders and followers across centralized and decentralized exchanges. A trader should be able to trade on one exchange while followers copy the same trade logic on their own supported exchange accounts.

- Exchange-independent trade mirroring
- Near 1:1 trade performance replication through proportional sizing
- Transparent trader performance and public discovery
- Fast event-based execution
- Premium Web3-native UX and branding
- Operational visibility for admin review and dispute handling

3. Product Goals and Non-Goals

Goals	Non-Goals for V1
Launch a stable multi-exchange copy trading core	Spot trading support
Mirror new trader positions automatically	Complex smart-contract subscription billing
Provide transparent trader performance data	Automated legal/compliance tooling
Support wallet-based crypto subscription payment	Unlimited exchange coverage
Deliver admin visibility and operational controls	Advanced social or community features

4. Roles and Permissions

The platform supports five user states or roles. A registered account can evolve from a standard user into a follower, a trader applicant, or both, while admin access remains separate.

Role	Core purpose	Main permissions
Guest	Public visitor	View landing page, browse trader overview, inspect public performance
Registered user	Account without active subscription	Manage profile, connect wallet, store exchange API keys, apply as trader
Follower	Subscribed user copying one or more traders	Subscribe, set copy preferences, view copied trades, receive alerts
Trader	Approved signal source	Connect trader exchange, publish profile, expose performance, generate copy events
Admin	Platform operator	Review traders, monitor users, inspect deviations from defaults, manage billing exceptions, view performance and failures

5. User Flows

5.1 Guest Flow

- User visits the website.
- User can browse the landing page and a public trader dashboard.
- User can inspect trader statistics and delayed trade history.
- User can register or log in.

5.2 Registered User Flow

- After registration or login, the user starts as a standard account without an active follower subscription.
- The user can connect a billing wallet.
- The user can add exchange API keys for future copy trading.
- The user can subscribe to a trader and become a follower.
- The user can apply as a trader.

5.3 Trader Application Flow

- User submits trader profile details except KYC.

- User connects trader API keys for supported exchanges.
- Application enters manual review by admin.
- Admin can set status to received, in review, rejected, approved, paused, or banned.
- Once approved, trader dashboard and public listing become active.

5.4 Follower Flow

- Follower connects exchange API key(s) with trading permission only.
- Follower selects a trader to copy.
- Follower pays for a subscription period per trader.
- Follower configures allowed exchange mapping, optional multipliers, slippage, and optional risk overrides.
- Only new trades opened after the follow starts are eligible for copying.

5.5 Admin Flow

- Admin can inspect users, traders, subscriptions, performance, and failures.
- Admin cannot directly log into user accounts through the same UX; admin uses dedicated oversight views.
- Admin can review whether follower defaults were changed, what was changed, and when.
- Admin can investigate support disputes with copied trade data, failed order logs, notifications, and billing state.

6. Page Structure and UX Scope

The frontend should follow a premium Web3/trading visual language: dark surfaces, bronze/gold accents, strong contrast, clean modern layout, and a trustworthy professional feel.

Page	Audience	Purpose
Landing page	Public	Project presentation, value proposition, CTA, sample trades, metrics
Guest dashboard	Public	Public trader rankings and headline performance
Trader overview	Public	Filterable list of all visible traders
Trader detail page	Public / follower	Detailed trader profile, metrics, delayed trade history
Registration / login	Public	Account access
User dashboard	Registered user	Starting point before follower or trader state
Follower dashboard	Follower	Performance, subscriptions, copied trades, failures, notifications
Trader dashboard	Trader	Own stats, profile, exchange linkage, subscription base
Billing page	Follower / user	Subscription purchase and renewal
API key management	User / follower / trader	Manage exchange connections safely
Notification settings	User / follower / trader	Telegram and in-app preferences
Admin panel	Admin	Moderation, operations, errors, billing and health views

7. Trader Discovery and Public Statistics

Public discoverability is a core acquisition and conversion mechanism. Guests should be able to browse traders and compare core performance metrics before registering.

- Public visible performance windows: 7 days, 30 days, 90 days, 1 year, all time

- Core visible metrics: ROI %, Winrate, Profit Factor, Max Drawdown, Number of Trades
- Default sort: percentage profit
- Alternative sorting: Winrate, Drawdown, Number of Trades, ROI, Profit Factor
- Visible delayed trade history on trader profile: 2-day delay
- Trader categorization: average trade duration, BTC/ETH focus vs altcoin focus

8. Subscription and Commercial Model

Commercially, the platform uses a simple fixed subscription fee per trader. The first release deliberately avoids free trials and complex revenue-share automation.

- Price per trader: USD 50 per month equivalent
- Supported durations: 1, 3, 6, 12 months
- Payment method: crypto only in V1
- No free trial
- No follower cap per trader in V1
- User pays separately for each trader they want to copy
- Revenue split across project, developers, and trader remains configurable in operations
- No refund flow in V1; access remains active until paid term ends

9. Copy Engine Logic

The copy engine is the heart of the platform. It must detect trader actions as fast as possible, normalize them, map them to follower constraints, and execute them on the follower exchange when eligible.

9.1 Copy Trigger Rules

- Execution model: event-based and as close to real time as possible.
- Only trades opened after the follow relationship starts are eligible for copy.
- Open positions that already existed before the follower started are ignored by default.
- Long and short positions are supported.
- V1 market scope: perpetual futures only.

9.2 Position Sizing

Default copying uses proportional sizing based on account balance. This is intended to reproduce trader behavior and achieve near 1:1 relative performance.

Example: if a trader has a USD 10,000 account and opens a USD 1,000 position, the trader uses 10% of account balance. A follower with a USD 1,000 account should therefore open a USD 100 position under default settings.

- Default mode: percentage-of-balance copy
- Follower override options: multiplier, max leverage, max stop loss %, max take profit %
- Any override must be protected by an explicit warning that copied performance can diverge from trader performance

9.3 Order Types and Trade Lifecycle

- The target product requirement is to copy all relevant trading actions.
- This includes: market orders, limit orders, stop loss, take profit, trailing stop, partial close, leverage changes, reduce-only, and post-only instructions.
- Where an exchange-specific instruction is not supported by the destination exchange in V1, the system should fail safely and notify the user rather than improvise silently.

9.4 Eligibility and Rejection Rules

Condition	System behavior in V1
Follower margin insufficient	Do not open trade; create notification and log failure
Asset unavailable on follower exchange	Do not open trade; create notification and log failure
Minimum order size or notional not met	Do not open trade; create notification and log failure
Required slippage threshold exceeded	Reject copy in V1; create notification and log failure
API or exchange execution error	Do not open trade; retry only if explicitly supported; notify and log failure

9.5 Exchange Mapping Rules

- If the same exchange is connected and valid, use that exchange first.
- If multiple follower exchanges are connected, use the first eligible exchange unless the user made an explicit selection.
- If the asset is not tradable on the target exchange, no trade is opened.
- Leverage should be mirrored 1:1 where possible.
- If follower-defined leverage caps apply, leverage is recalculated within the allowed limit.

The concept assumes straightforward symbol normalization for core crypto assets and perpetual markets. The implementation should still include a normalized exchange symbol registry to avoid hidden naming or precision mismatches.

9.6 Slippage Handling

- Default slippage threshold: 1%
- Follower may change the slippage setting
- If slippage exceeds threshold in V1: reject the trade
- Potential V2 extension: optional conversion to limit-order retry logic

10. Recommended Technical Architecture

The technical architecture should be opinionated rather than left open-ended. This platform is fundamentally an event and order orchestration system, not just a web dashboard.

10.1 Stack Recommendation

Layer	Recommendation	Reason
Frontend	Next.js + TypeScript	Good fit for public pages plus authenticated dashboards
Backend API	Python + FastAPI	Fast API development, strong async ecosystem, good worker integration
Primary database	PostgreSQL	Reliable relational core for users, subscriptions, trades, logs, reporting
Cache	Redis	Fast transient state, caching, rate limiting, live dashboard support
Event / message bus	NATS JetStream	Persistent event handling and replay-friendly trade orchestration
Realtime updates	WebSockets	Push dashboard updates, statuses, and notifications
Infra packaging	Docker	Portable deployment and environment consistency
Monitoring	Prometheus + Grafana	Operational visibility
Centralized logging	Loki or OpenSearch stack	Failure analysis and operations review

10.2 Service Layout

- Web application / frontend service
- Backend API service
- Copy engine worker service
- Exchange adapter services or modules
- Billing and wallet verification service
- Notification service
- Admin reporting and analytics service

Separating the copy engine from the general API is critical. The user-facing application and the trade execution pipeline should not compete for the same runtime resources under load.

10.3 Core Event Flow

- Trader action is observed from source exchange.
- Trade event is normalized into a platform-internal event format.
- Event is published onto the message bus.
- Copy engine resolves eligible followers.
- Follower exchange mapping and symbol eligibility are checked.
- Position size, leverage, slippage, and user overrides are evaluated.
- Execution request is sent to the target exchange adapter.
- Result is written to persistent trade records and failure logs.
- Notifications are emitted to in-app and Telegram channels.

11. Data Model and Database Structure

The exact schema can evolve, but the platform should be designed around a clear relational model from day one.

11.1 Core Entities

Entity	Purpose
users	Master user account record
wallet_bindings	Connected billing wallet(s) and verification state
roles	Effective role flags or derived entitlements
trader_profiles	Public trader data and approval status
follower_profiles	Follower preferences and default copy settings
exchange_accounts	User-connected exchange accounts by role and exchange
api_credentials	Encrypted credentials metadata and secure references
subscriptions	Follower-to-trader billing relationship and term
copy_relationships	Active copy configuration between follower and trader
trade_events	Normalized trader-side events
copied_trades	Per follower execution outcomes
trade_failures	Execution failures with categorized reasons
performance_snapshots	Aggregated metrics for public and private

	dashboards
notifications	Stored notification history and status
telegram_bindings	Telegram chat or account linkage
audit_logs	Security, settings, and admin activity logs
billing_orders	Pending and completed subscription purchase intents
billing_transactions	Onchain or later-provider verified payment records

11.2 Critical Audit Fields

- created_at, updated_at, deleted_at where relevant
- created_by, updated_by for admin or internal actions
- effective_from and effective_to for subscription periods
- change_source for whether a setting came from default, user override, or admin action
- settings_changed_at to support dispute review

12. Notification System

Notifications are operationally important, not cosmetic. They provide transparency when trades execute, fail, or diverge from expectations.

- Channels in V1: in-app and Telegram
- Follower can enable or disable notification types individually
- Suggested events: trade opened, TP hit, SL hit, trade not copied, API error, insufficient margin, subscription expiring, trader paused, trader removed, high drawdown alert

The notification service should subscribe to trade and billing events instead of being tightly embedded inside the copy engine. This keeps the execution pipeline cleaner and makes later channel expansion easier.

13. Billing and Wallet Security Concept

V1 billing should be crypto-native but intentionally constrained. The platform should not begin with a complex smart-contract billing layer. Instead, billing should combine wallet binding, payment order generation, and verified subscription activation.

13.1 Billing Principles

- Wallet connection required before payment
- Payment must originate from the connected wallet
- Billing order is tied to one wallet, one trader, one duration, one amount, and one validity period
- V1 should use one stablecoin and one supported chain only
- Subscription activates after verified payment confirmation
- Manual intervention should be exception-only, not the standard flow

13.2 Suggested Billing Flow

- User connects wallet and signs a wallet-binding message.
- User selects trader and subscription duration.
- System generates a billing order with exact payment details.
- User submits payment from the bound wallet.
- Billing service verifies wallet, token, chain, amount, and receiving address.
- If valid, subscription activates immediately.
- If payment cannot be matched cleanly, the case is moved to admin exception handling.

13.3 Wallet and Session Security Requirements

- Use signed wallet-binding messages with nonce, issue time, and expiration
- One-time nonce usage only
- Bind session to both user account and verified wallet
- Require re-verification for sensitive billing changes if needed
- No blanket or unlimited token approvals required for V1 billing
- Clear UI language showing exactly what is being signed and what is being paid

14. Exchange API Security Requirements

API key handling is one of the most sensitive parts of the platform and should not be treated as a casual implementation detail.

- API keys must be encrypted at rest
- Secrets should be separated from general application configuration
- API keys must never be shown in full after storage; show masked values only
- Server-side access only; frontend must never receive raw secrets
- Withdraw permission is forbidden
- Trading permission is required where applicable
- Automated checks should warn or reject keys with unsafe permissions
- Audit log every add, update, rotate, disable, or delete action
- Sensitive values must never appear in normal application logs

15. Admin and Operations

Because the platform starts with a single admin operator, admin tools should focus on visibility, not enterprise bureaucracy.

- User overview
- Trader overview and approval workflow
- Follower subscription overview
- Performance inspection
- Failure and retry log review
- Billing exceptions and payment verification view
- Notification delivery status
- Exchange health and API health view
- Revenue overview
- Support and dispute handling context

16. Reporting and Transparency

- All-time historical trade storage
- CSV export support
- Follower-side and trader-side trade history
- Equity curves
- Monthly reports

Reporting should serve both marketing and dispute resolution. Public statistics help conversion; private historical records help support and trust.

17. Hosting and Infrastructure Recommendation

- V1 deployment can start on Hetzner Cloud for cost efficiency
- AWS can be adopted later if scale, compliance, or managed service needs increase
- Use Dockerized services from the beginning
- Keep PostgreSQL backups automated and tested
- Separate production, staging, and local development environments
- Use centralized monitoring and logs from the first usable version

18. V1 / V2 Roadmap

V1	V2 / later expansion
Perpetual futures only	Spot support
Hyperliquid, Aster, BloFin	Additional exchanges
Default slippage rejection	Optional limit retry after slippage rejection
Crypto-only billing with one chain and one stablecoin	Managed payment provider integration

Telegram + in-app notifications	Additional channels such as Discord, email, push
Manual trader approval	Expanded internal operations roles

19. Final Implementation Notes for Developers

This product should be implemented as an event-driven trading orchestration platform with clear separation between user-facing application logic, exchange integration logic, billing, and notifications.

- Do not treat the copy engine as just another API endpoint.
- Do not store exchange secrets casually or expose them through the frontend.
- Do not use manual billing handling as the long-term baseline architecture.
- Build all critical user changes, trade failures, and billing state changes with auditability in mind.
- Prefer explicit safe failure over silent assumptions when a follower exchange cannot faithfully mirror a trader action.

20. Open Configuration Items

The platform concept is complete enough for engineering planning, but the following operational values remain configurable rather than unresolved:

- Revenue split percentages between project, developers, and trader
- Chosen chain and stablecoin for V1 billing
- Exact ranking formulas beyond current sortable metrics
- Exact retry policy for transient exchange failures
- Later exchange onboarding sequence after V1